

Problem 1

A) In normal logging in programs, there are attempt rate limits, which would become useless should the hashes are not protected.

B) I suppose there is no real gain in limiting which hash function to use for all users' passwords, except that it will make the system design simpler. In contrast, if the hash functions are unlimited, the system can be easily upgraded in the future with safer hash algorithms (without the user's even knowing about it!)

C) Making a hash evaluation much slower, thus limiting brute force attacks.

D) I would set a reasonable count that can make it harder at least 1000 times for an attacker, while not enough to cause big slow down to the server (ideally, maybe 100th of a second for one authentication), since the server is popular. If 100 people are using the server together, that will take a total of 1 second of CPU time, which is big already.

E) I will use a different strategy on my personal laptop, because only me will be using it, and I don't care if I will wait for a few seconds for one mistyped password, I will set the count so that it will take at least seconds to calculate such a hash.

Problem 2

B) Number of total non-empty strings $\Sigma^{\leq n}$ is 26^{20} . By the birthday paradox, the number of average tries is 26^{20} .

D) Since each salt is unique, the cost would change to be as if we only have a single hash target at a time (ignoring collisions). That is, it would be slowed by the factor of the number of hashes in `hashes.txt`.

Problem 4

A)